DEVELOPER HANDOVER · BUILDER'S EDITION

SupportOS+

The Handover Book — why the code is the way it is. Every architectural decision, every rejected alternative, every sharp edge, written for the programmer who inherits this project.

32 CHAPTERS | 7 PARTS | WHY · WHY NOT · WHAT BREAKS | THE LAWS OF THE CODEBASE | SHARP EDGES A–Z

VERSION 1.8.0

FOR THE NEXT MAINTAINER

ABOUT THIS BOOK

How to use the Handover Book

This book exists because at some point, every project changes hands. The programmer who takes over SupportOS+ will read the code and, within a week, start asking questions that the code alone cannot answer: *why is the UI hand-drawn in egui instead of a web stack? Why is there no ORM? Why does the sandbox kill syscalls with EPERM instead of killing the thread? Why are prompts hard-coded? Why is there no CSV export? Why was the git history rewritten?* Those questions have answers. Most of them were conscious decisions, made under real constraints, often after a specific bug. This book records them.

How it was written. Four research passes went through every crate of the workspace — `crates/catalog`, `crates/core`, `crates/app-shell`, `crates/xtask` — plus the build scripts, CI workflows, and the in-repo decision ledgers. Every claim carries a file-and-line reference, and where a rationale exists as a source comment, the comment is quoted. The repo's own decision documents (`ai/decisions.md`, `ai/plan.md`, `ai/completed.md`, `repo-audit-2026-10-24/`) are cited where they explain more than the code does. Nothing here is guessed: where the code is silent, the book says so.

> **The notation you will meet everywhere**
>
> Source comments are full of markers like `Clean-audit item 74`, `Audit fix (M12)`, `A1` or `D4`. These are **historical decision records**: numbered findings from the 2026 audit and clean-audit passes, each of which changed the code. They are not noise — they are the *reasons*. When this book says *item* 74, it means: look for the comment, read the war story behind it. The full ledgers live in `ai/completed.md` and `ai/plan.md`.

How the chapters are organized. Each decision is presented in the same rhythm: *the decision*, *the evidence* (where in the code), *why this way* (the engineering rationale, including the constraints that forced it), *the rejected alternative* (what a newcomer would assume is "the normal way", and why it was not taken), and *sharp edges* (what breaks if you change it without reading first). Part VII compresses all of

it into a law list, an A–Z sharp-edge index, and a register of known trade-offs and dormant paths.

If you read only three chapters before touching anything: Chapter 7 (*Migrations and schema law* — the one rule that has already caused an outage when broken), Chapter 30 (*The laws of the codebase*) and Chapter 31 (*Sharp edges* A–Z). Then read the rest at leisure — preferably with the repo open beside you.

Table of contents

PART I — READ ME FIRST

1 What this book is . . . 1
2 The project in one sitting . . . 2
3 Four crates, one-way ladder . . . 4

PART II — THE MACHINE ROOM

4 The boot sequence . . . 5
5 Threads and the event bus . . . 7
6 The database layer . . . 9
7 Migrations and schema law . . . 11
8 Errors, panics and toasts . . . 12
9 Storage, config, single instance . . . 14

PART III — SECURITY & DATA SAFETY

10 The sandbox . . . 15
11 Secrets and the envelope . . . 16
12 Backups and restore . . . 18
13 Bundles, resets, secure delete . . . 19
14 Webhooks and OAuth . . . 20
15 Untrusted content . . . 22

PART IV — THE AI STACK

16 Local-first AI . . . 23
17 Five roles and the golden set . . . 24
18 Guardrails . . . 25
19 The read-only agent . . . 27
20 Vectors and hybrid search . . . 28
21 The ledger and the prompts . . . 29

PART V — PRODUCT LOGIC

22 Automation . . . 31
23 SLA and workload . . . 32
24 Reports and segments . . . 33
25 Outreach and incidents . . . 34

PART VI — BUILD, RELEASE & REPO LAW

26 The daily build . . . 35
27 Packaging . . . 36
28 The release pipeline . . . 38
29 The history rewrite . . . 39

PART VII — LIVING IN THE CODE

30 The laws of the codebase . . . 40
31 Sharp edges A–Z . . . 43
32 Known trade-offs, open ends . . . 44

What this book is

The contract between the outgoing and the incoming maintainer, and the rules this book follows.

A handover document usually fails in one of two ways: it is a fanfare ("world-class architecture, best practices everywhere") that tells the newcomer nothing, or it is a dump of tribal lore with no evidence. This book tries for a third thing: **every architectural decision, paired with its evidence, its rationale, the alternative that was rejected, and the damage you can do by ignoring it**. Where the honest answer is "this is a trade-off that could have gone the other way", the book says that too. You are a programmer; you deserve arguments, not slogans.

1.1 The sources, and only the sources

Everything in this book was extracted from exactly three layers of the repository, in this order of authority:

1. **The code itself** — the four crates under `crates/`, the CI workflows, `scripts/`, `deny.toml`, `rust-toolchain.toml`. Constants, pragmas, limits, fallback chains: these are facts.
2. **Source comments as decision records** — comments marked `Clean-audit item N`, `Audit fix (MN)`, `Item N`, `A1/A2`, `R5/R7`, `D1-D5`. These record *why* a line looks the way it does, usually by describing the bug it once was.
3. **The in-repo ledgers** — `ai/decisions.md` (the port's decision ledger), `ai/plan.md` and `ai/completed.md` (the clean-audit ledgers), and `repo-audit-2026-10-24/` (the external audit, including the history-rewrite record).

Where the book could not find a written rationale, it reconstructs one from the constraint set (solo maintainer, always-green main, offline-first, 4 GB-class build machines) and marks it as reconstruction. Two popular rumors were checked against the source and *corrected*: there is no `-j2` build flag and no thin/fat LTO split anywhere in the repo (Chapter 26), and the release smoke is `SPP_SMOKE=1` with the container's own `HOME`, not a scratch `XDG_DATA_HOME` (Chapter 28). When lore and code disagree, the code wins — that discipline is the whole spirit of this project.

1.2 What this book is not

It is not a user manual — that exists already, also in this repository (`manual/`), written for non-technical users. It is not a line-by-line code tour; it deliberately skips anything that is obvious from the source. And it is not a roadmap: Chapter 32 lists open ends and dormant paths, but the decisions about what to build next are yours.

1.3 Conventions

- File references like `crates/core/src/db.rs:80` are exact at the time of writing (v1.8.0 line). Line numbers drift; file names and comment markers do not.
- Identifiers, settings keys and syscalls appear in mono type: `with_write`, `secure_delete`, `PRAGMA foreign_keys`.
- Boxes titled *Sharp edges* are warnings: changes that broke something once before, or that will break something specific and predictable.
- Boxes titled *Rejected* record the alternative that was considered and refused — read these before proposing the same idea again.

CHAPTER 2 · PART I

The project in one sitting

What SupportOS+ is, where it came from, and the two sentences that explain almost every odd decision.

The product. SupportOS+ is a Linux desktop application that mirrors a company's Help Scout help desk into a local SQLite database and works on that mirror: 23 pages of inbox, customers, knowledge, reports, automation and optional AI, all instant and offline because the data is local. Writes go out to Help Scout (or wait as explicit, visible outbound jobs); reads never leave the machine. There is no server component, no account system, no telemetry, no auto-update. One binary, one data directory.

2.1 The origin story you must know first

This repository is a **port**. The original program (pre-1.0.0, called SupportOS++) was a web-technology desktop app: Tauri 2 for the window, Leptos/WASM for the UI, and an axum HTTP server on loopback glueing the two to a pure-Rust business core. The port — documented decision by decision in `ai/decisions.md` — replaced the entire presentation and transport stack with a native **egui** shell while *keeping and hardening the roughly 90,000-line pure-Rust core*. The scale of the cutover is quantified in the ledger: about 33,600 lines of Leptos UI and 21,000 lines of axum

handlers were deleted; roughly 245 of 345 database lock sites died with the router. The old stack's scar tissue is all over the codebase in comments that begin "the web route did X…".

Two consequences follow, and they explain half of this book. First, **the core predates the shell**: its synchronous `&Connection` API shape was kept, and the async world is confined to small, documented seams (Chapter 5). Second, **every oddity that looks like nostalgia is usually a migration-map row**: a bug in the old stack that the port explicitly fixed, listed in `ai/decisions.md` §3 (rows 1–16) and cited in code comments.

2.2 The goal stack, verbatim

`ai/decisions.md` states the project goals in strict priority order: **Stable → Easily maintainable → Full feature parity → Future development friendly**, with the declared non-goals: cloud sync, telemetry, cross-platform packaging (Linux AppImage/.deb only), and mobile. This ordering is not decoration. It is the tie-breaker for every trade-off in the book: stability beats maintainability, which beats feature parity, which beats extensibility. When you are tempted to restructure something "properly", ask which goal you are buying and which you are selling. Most refactoring temptations sell *Stable* to buy *Future development friendly* — the project has consistently ruled against that trade, on purpose.

2.3 The two founding honesty laws

Everything else in the culture reduces to two laws, both enforced by CI:

- **Nothing is simulated.** The app boots into exactly two states — `Real` (credentials resolved) or `Empty` (an inert provider serving nothing). The pre-1.2.0 simulated demo account was deleted so thoroughly that migration m050 drops its settings rows "for good", and `scripts/gate_no_simulation.py` runs *first* in CI, grepping production code for the markers `simulated`, `fake`, `demo`. A support tool that shows invented tickets trains its users to distrust everything it shows.
- **Nothing is silent.** Errors surface three ways at once (toast, durable diagnostics table, fatal log — Chapter 8); degraded features print *why* they degraded (search mode notes, capability banners); toasts report honest outcomes of destructive actions. The audit's recurring theme is converting "silently synced 0 collections" into "a labeled error".

Why this matters to you specifically

You will be tempted, in your first month, by at least three of the rejected alternatives in this book — they are all reasonable ideas that this project refuses for reasons that were expensive to learn. The cheapest way to inherit the project is to assume every weird thing is weird *on purpose*, find its chapter here, and only then decide.

CHAPTER 3 · PART I

Four crates, one-way ladder

Why the workspace is shaped like catalog ← core ← app-shell, plus xtask — and what each boundary buys.

The workspace (`Cargo.toml:3-8`) has exactly four members, and their dependency direction is a one-way ladder: **catalog** depends on nothing but serde; **core** depends on catalog; **app-shell** depends on core; **xtask** depends on core and is depended on by nobody. A new developer's first question is usually "why is *catalog* a separate crate? It's just enums."

3.1 Catalog is a closed vocabulary, not a library

`crates/catalog` holds every user-visible domain *vocabulary*: the 21 report metrics and 14 dimensions, the 12 graph node kinds, incident statuses and severities, custom-field types, operations tiles, copilot tools. Each is a closed enum with a `pub const ALL` array and lookup functions. Its crate description states the contract: "*single source of truth for DB, validation, UI — WASM-safe (no I/O deps)*", and its `lib.rs:1` begins with `#![forbid(unsafe_code)]`.

The payoff is mechanical: adding one variant to a catalog enum produces **compile errors in every database query and every UI page that forgot the new kind**. The compiler, not a runtime test, enforces the closed-world assumption — a page cannot invent a metric name that no query implements, because the page iterates `ReportMetricKey::ALL`. This is why the Reports builder and the metric catalog can never drift apart.

3.2 Why core is UI-free, and why it must stay that way

`crates/core` (~90K lines) is the business logic, SQL, sync engine, AI stack and vector store. It knows nothing about egui. This is what made the port survivable: the Tauri shell and the egui shell coexisted on the same `main` during the side-by-side cutover period, because the core never imported either. It is also what makes the

700+ unit tests fast and windowless: `cargo test --workspace` never links a GUI stack. The rejected alternative — the newcomer's default "core + api + web" split — was exactly the old architecture, and the port deleted it.

3.3 Where code belongs

YOU ARE ADDING…	IT BELONGS IN…	NEVER IN…
A domain word (status, kind, metric)	`catalog`, as an enum + ALL array + label	`core` as a string constant — that drift
SQL, sync, AI, storage logic	`core`, in the module owning the tables	`app-shell` — the shell is glue only
A page, widget, toast, dialog	`app-shell`, calling core write APIs	direct SQL in the shell (bypasses `with_write` law)
Build/package/CI helper	`xtask` — the developer entry point	anything shipped to users

Sharp edges

- Never add an I/O or GUI dependency to `catalog` — its no-I/O contract is what lets it be reused by future tools untouched.
- Never introduce a dependency cycle through dev-dependencies; the ladder is load-bearing for the test setup.
- `xtask` is invoked via the `cargo xtask` alias (`.cargo/config.toml`) and is **not shipped** in either package.

CHAPTER 4 · PART II

The boot sequence

A 190-line main, a strict boot order, a fatal path designed for windowless failures, and the smoke mode that renders every page.

`crates/app-shell/src/main.rs` is about 190 lines and its doc comment lists the exact boot order: logging → single-instance lock → database with all boot migrations → core event bus → boot services (sync engine + provider, real or empty, hot-swappable since 1.4.0) → background worker host → minimal loopback listener → eframe window. The comment that follows is the design law: "*Everything heavy lives in the library so it stays unit-testable without a window.*"

4.1 Why main is thin (decision D5's practical form)

The binary is a boot script; every subsystem is a `lib.rs` item. This is what allows the smoke harness (below) and the 300+ page tests to construct the *real* shell against a bare `egui::Context`. It also gives the codebase exactly one legitimate home for `process::exit(1)`: the fatal boot paths. A fat `main` would make all of that impossible; the Tauri-era equivalent was 118 lines of untestable glue, and nobody missed it. If you are about to add anything to `main` beyond the listed duties (init logging, migrate legacy data dir, install panic hook, acquire lock, open DB, apply log level, build boot services, start workers, bind loopback, run smoke-or-window), it belongs in `lib.rs` instead.

4.2 The fatal path: failures that happen before a window exists

`crates/app-shell/src/fatal.rs:1-9` explains itself: "*The desktop entry discards stderr, so on a GUI launch every failure in that window-less phase used to vanish without a trace ('opened it, nothing appeared').*" Boot-phase failures and panics now land in `logs/fatal.log` (with version, OS, session type, and the resolved `SPP_RENDERER`) and, best-effort, in a `zenity` / `notify-send` dialog. The log writer is best-effort by law — "*diagnostics must never crash the crash path*" — and a test pins that it survives an unwritable data dir. The panic hook (installed in `main`, v1.1.1) routes renderer and context panics into the same file.

4.3 SPP_SMOKE: the binary proves itself (clean-audit item 71)

Set `SPP_SMOKE=1` and, after the full boot chain, the binary constructs the real `ShellApp` against a plain `egui::Context` and renders one full pass of every `PageKey`, asserting each frame painted non-zero shapes, then exits 0. The comment records why: the smoke "*used to return after the boot chain ... and BEFORE the shell/renderer existed — a renderer regression could not fail CI because the binary under test never rendered a frame.*" This is the tool that makes GUI CI possible at all: no display server, no GPU, seconds of runtime, exercising the actual shipped artifact — which is why the release pipeline runs it inside the built .deb payload and the extracted AppImage (Chapter 28). Note the app has **zero CLI arguments by design**; `SPP_*` environment variables are the only runtime switches, and the smoke check is strictly `== Ok("1")` so it can never activate accidentally.

Sharp edges

- A new page is automatically smoke-covered (the harness iterates `PageKey::all()`) — but a page that can legitimately paint zero shapes would fail the smoke. Keep the paint contract.
- The smoke shares `ShellOptions`; anything it starts (workers) must shut down on return.
- Do not add a `--smoke` CLI flag: CLI args would be a second config surface; env vars are the contract.

CHAPTER 5 · PART II

Threads and the event bus

One worker thread hosting a two-slot tokio runtime, a bridge that turns broadcasts into repaints, and the deadlock essay every new hire should read.

The concurrency model has three pieces: a **worker host** that owns all background timers, an **event bus** that carries every engine's progress to the UI, and a set of **documented seams** where the synchronous core touches async HTTP. Each piece exists because a specific alternative was tried or seriously considered and rejected — usually with the failure mode written down.

5.1 The worker host: one thread, two workers, owned by main

`crates/app-shell/src/workers_host.rs` spawns a plain OS thread that builds a tokio multi-thread runtime with exactly `WORKER_THREADS = 2`. The comment justifies the number: two worker threads "cover the overlapping long-poll timers (sync + connector refresh) while keeping the idle footprint minimal on desktop machines." Nothing in this app is CPU-bound; it is network + SQLite. The host is owned by `main`, *not* by `ShellApp` — deliberately, so that page tests construct the shell with `workers: None` and never see a background timer fire mid-test. A startup handshake (mpsc readiness + 30-second timeout + oneshot shutdown) and a RAII `Drop` make shutdown deterministic. When the user changes the sync interval, the settings page calls `WorkersHost::restart()`, which re-spawns onto the worker runtime's handle — because `WorkerManager::start` uses `tokio::spawn` and "the settings page runs on the UI thread".

5.2 The bridge: push before repaint

Immediate-mode egui paints no frames without input, so a UI that polls a channel inside `App::ui` would starve. The design (`crates/app-shell/src/bridge.rs:1-15`)

instead runs a dedicated OS thread blocked on the core `EventBus` broadcast channel (capacity 256, `eventing.rs`); every arriving `ServerEvent` is queued into a `Mutex<VecDeque>` *and then* `request_repaint()` is called. The ordering is the correctness argument: "the queue is pushed before the repaint fires, so a woken frame is guaranteed to observe the event that triggered it." A lagged subscriber is logged and bridged past (the bus is bounded by design); a poisoned queue mutex is recovered, not propagated. Because repaints only happen on input plus these wake-ups, `app.rs` also runs a 1.5-second repaint heartbeat — added after live "sending…" status lines were observed never refreshing. Which pages a given event refreshes is decided by `pages::invalidated_pages(event)`; adding an event type without registering it there means pages silently stop refreshing.

5.3 The seams: where async meets the synchronous core

The core's ~345 legacy call sites take `&Connection` and are synchronous. The port's rule (`ai/decisions.md` D3) was: closures over connections are short and sync; futures are driven to completion on runtimes spawned *per job*; results flow back through the event bus. Three concrete patterns:

- **Manual sync actions** run on detached, named `std::thread`s with their own single-thread tokio runtime; terminal state arrives via `SyncCompleted` on the bus, never via the dropped thread result (`sync_health.rs:188-197`).
- **AI pipelines** drive their futures inside `core/src/workers.rs:26-52`'s `block_ai_pipeline`, on a *private current-thread runtime*. The comment is a small essay titled, in effect, "why not `Handle::current().block_on`": every AI runner holds the DB write guard for the whole pipeline call, and parking the only thread of a current-thread runtime against that same write mutex "is a guaranteed deadlock for the integration tests … and a production stall: every DB write in the app would freeze for the whole LLM round-trip."
- **Long LLM work** takes the DB through `DbHandle::long_op_guard()` (`db.rs:214-229`), which opens a fresh tuned connection instead of pinning the exclusive writer lane across network round-trips.

Rejected: async everywhere

The "normal way" would be tokio end-to-end with `spawn_blocking` and an async DB layer. That would have forced rewriting every sync module during the port and multiplied deadlock surfaces — the `block_ai_pipeline` comment documents what partial async adoption without thinking already cost. The full-async rewrite remains available to you; the book's only demand is that you read the essay first.

Sharp edges

- Never call `Handle::current().block_on` from a thread that holds any DB guard.
- Never hold `write_guard()` across an `.await` or a network call — use `long_op_guard()`.
- Page code never blocks; spawn and report via the bus. Push events *before* repainting.
- Every new `ServerEvent` must be added to `pages::invalidated_pages`.

CHAPTER 6 · PART II

The database layer

One writer, two WAL readers, four pragmas, zero ORM — the shape of every data access in the app.

All code — UI and background — holds a cloneable `DbHandle`, never a raw connection. For file-backed databases it wraps one exclusive writer connection (guarded by a mutex) plus two dedicated reader connections; in-memory databases degrade to a single shared lane. The constant is documented at `db.rs:80-101`: SQLite WAL permits exactly one writer at a time, so "serializing it with its own mutex is the same contract an OS writer thread would enforce, without forcing every write closure to be `'static`". The two readers serve concurrent page reads while a write is in flight; acquisition `try_lock`s across the pool and falls back to a blocking lock on reader[0] "because the pool only has to absorb bursts".

6.1 Why a facade, and why the pool came late

The facade is a deliberate seam from the port plan (decision D3): the shell "binds to a `DbHandle` facade wrapping the current `Arc<Mutex<Connection>>`", so the pool could be introduced *after* cutover without touching the roughly 100 surviving call sites,

and the strategy can change again behind the same type. The pre-pool behavior — every read queuing behind every write — was real and measurable. Poisoned mutexes are recovered, not propagated (a panic inside one closure must not kill the app), and if no readers can be opened the handle degrades gracefully to the old single-connection behavior.

6.2 The pragma contract

Every connection path goes through `db.rs` `open()`, which sets exactly four things: `journal_mode=WAL`, `synchronous=NORMAL`, `foreign_keys=ON`, `busy_timeout(5s)`. Each is load-bearing: WAL is the read-concurrency story; `synchronous=NORMAL` is the durability/speed trade appropriate to a desktop app whose backup story is explicit export files; `foreign_keys=ON` matters because SQLite defaults it *off* — a raw `Connection::open` somewhere else silently drops referential integrity; the 5-second busy timeout absorbs writer contention between the UI lane, worker lanes and long-op connections instead of erroring. `rusqlite` is used with the `bundled` feature: zero system SQLite packages at build or run time, one self-contained AppImage, and pinned SQLite behavior everywhere — the same reason FTS5 and the sqlite-vec virtual tables are first-class citizens here and would be second-class under an ORM.

> **Rejected: diesel / sea-orm / sqlx**
>
> sea-orm and sqlx want async runtimes and bring their own migration tooling — a second migration source of truth that would collide with the Rust registry (Chapter 7). Diesel pulls a macro stack whose schema DSL would have to be kept in lockstep with dozens of hand-audited migration steps written by one person who reads SQL directly. Hand-written SQL keeps ~80 domain modules readable, diffable, and friendly to FTS5 and vec0 virtual tables, which ORMs fight.

6.3 The three lanes

- `with_read` — pooled reader lane; page rendering, queries, search.
- `with_write` / `write_guard()` — the exclusive writer; short transactions opened *inside* the closure.
- `long_op_guard()` — "for background work that spans network round-trips (LLM pipelines), so the exclusive UI writer lane is never pinned across them" (`db.rs:214-229`); it opens a fresh tuned connection on the same file whose writes serialize against the writer lane inside the shared busy timeout (audit A2).

Tests using `:memory:` silently run single-lane mode — never port a file-backed assumption into them. Reader connections are opened at `DbHandle::new` time and assume the schema is already migrated: wrap connections only after migrations run.

CHAPTER 7 · PART II

Migrations and schema law

No SQL files, no ORM tool: a two-layer Rust registry, one transaction per step — and the one rule that has already caused an outage when broken.

There are no `.sql` migration files. Versions 1–2 live in `core::migrations::MIGRATIONS` as inline SQL constants; versions 3–53 live in `core::bootstrap::BOOT_STEPS` as `fn(&Connection)` entries **owned by the module whose tables they create**. Each applied step is recorded in the `_migrations` table (version, label, applied_at) — chosen over `PRAGMA user_version` because labels and timestamps are for humans auditing an install. A step runs in its own transaction: a failed migration leaves the database at the previous version, never half-migrated.

7.1 Why migrations are Rust functions next to their modules

A feature PR adds its tables in the same file as its queries; the compiler links the two, and the audit history (the comments) is the documentation. The rejected alternative — sqlx/diesel migration folders — would create a second source of truth that drifts from module code, plus async tooling dependencies. Runtime DDL is banned outright: the webhook-events schema "used to be re-ensured on every insert; it now lives here, in its own numbered step (migration-map row 6)" (`bootstrap.rs:209-215`) — runtime `CREATE TABLE` incl. DROP was a confirmed pre-port defect.

7.2 The foreign-key dance

FK enforcement is suspended while a step runs, and `bootstrap.rs:346-356` explains why in full: several steps rebuild tables by dropping and renaming them, and "SQLite's live enforcement cannot survive that dance: immediate cascade actions fire against rows that are about to come back under the rebuilt table, and the deferred-violation counter never forgets a violation that the rename has long since repaired (the 1.1.x 'migration 36 failed' boot failures on real 0.2.x data)." Integrity is instead enforced by an explicit `PRAGMA foreign_key_check` sweep inside the same transaction: a step that would commit dangling references rolls back whole.

7.3 The war story that defines the law

> **Never edit an applied migration**
>
> `bootstrap.rs:320-332`, verbatim in spirit: "*customer_chats was originally delivered by editing the already-applied m039 in place. Databases that had run the released 1.7.3 recorded version 52, so every later boot skipped m039 and the customer chat-channel mirror never existed on upgraded installs — the first chat-channel sync failed with 'no such table'. m062 re-issues the same idempotent CREATE: a no-op where the current m039 already made the table (fresh installs), the only creator where it did not (1.7.3 → 1.8.0 upgrades).*" This is the law's proof: **_migrations already recorded the old step on user machines; editing it silently skips the change on every upgrade.**

7.4 The exact recipe for migration N+1

1. Append a new `BootStep { version: 54, label: "m063_…", apply: your_module::apply_m063 }` to `BOOT_STEPS`.
2. Implement `apply_m063(conn: &Connection) → Result<()>` next to the module's other schema code, using idempotent SQL (`CREATE TABLE IF NOT EXISTS`); if you are patching what an older release's step produced, mirror the m062 pattern.
3. Never modify an existing step's SQL, label, or version; never reorder; bump nothing else.
4. Remember the boot-order detail: the secrets master key is initialized *before* steps run, because m049-sealed envelopes must be openable (`bootstrap.rs:399-416`).

CHAPTER 8 · PART II

Errors, panics and toasts

One error enum, three surfacing channels, and a panic philosophy: contain, log, keep the lane alive.

`core::error::Error` is a single `thiserror` enum with a `Result<T>` alias — `Sqlite`, `Migration`, `Config`, `Io`, `SecretNotFound`, `WebhookSignature`, `WebhookDuplicate`, `OauthStateInvalid`, `Json`, `Validation`, `Other`. One enum keeps `?` working across

90K lines and gives the UI matchable variants. `anyhow` exists in the workspace but lives only in xtask, where context-free bubbling is fine.

8.1 Three channels for one failure

Background failures surface simultaneously to three audiences (`app.rs:694-706`): a **toast** for the operator sitting at the window (non-outbound `ErrorOccurred` events become warning/error toasts); the `application_errors` table — capped at 500 rows, rendered by the Diagnostics page — for the operator diagnosing later (`audit.rs:204-244`; levels are normalized so "anything else is stored as `error`, so a typo can never hide a failure"); and `logs/fatal.log` plus a desktop notification for the pre-window boot phase (Chapter 4). One outbound-failure toast is singled out in the source as "the single most important toast in the app: 'it did not go through' must never be silent again."

8.2 Toast policy is deliberate — do not "fix" it

Two audit items define behavior. Item 69: toast lifetime burns only across *rendered* passes (with a `MAX_PASS_GAP` of 1600 ms), because "a wall-clock deadline would expire toasts the user never saw: an error raised while the window was hidden vanished silently the moment it was shown again." Item 23: `None` means "say nothing" — background auto-syncs invalidate pages silently; only manual runs produce toasts. Reverting either looks like a cleanup and reintroduces a fixed bug.

8.3 Panic discipline

A long-running desktop process cannot treat a panic as process death (the user loses their window and their in-flight reply) nor as a free pass. Hence: every worker task runs inside `contained(name, fut)`, which logs a panic to `application_errors` under the task's name and keeps the lane alive; every timer tick is guarded by a RAII `TickGuard`; DB/queue mutex poisoning is recovered at the lock helper. The TickGuard exists because of bug A1: "a mid-tick stop … or a contained panic stranded the flag in the set state forever — every future tick of that timer then returned instantly and the job queue (or the sweep) silently died until process restart." Similarly, the stale-job sweep consults `executing_jobs` + `boot_unix` (audit M16) so a long-but-alive job "can never be re-queued mid-run — the old sweep double-executed anything running past 30 minutes, duplicating sends." `panic = "abort"` was rejected for exactly the window-loss reason; `.unwrap()` in prod paths is reserved for provable invariants or poison-recovering lock helpers — treat every other one as a bug.

Storage, config, single instance

One directory owns everything; environment variables are the only configuration; a flock keeps the second instance out.

9.1 The data directory

Default location: `$XDG_DATA_HOME/supportos-plus` (falling back to `$HOME/.local/share/supportos-plus`), overridable by `SPP_DATA_DIR`. The layout (`config.rs`): `supportos-plus.db` (+ WAL/SHM), `secrets.key` (0600), `vectors/vectors.db`, `backups/`, `attachments/`, `logs/fatal.log`, `instance.lock`. One directory is what makes "backup = copy one folder", atomic restore, and the 0.2.x legacy move simple. That move (`config.rs:43-67`) is a model of data-safety prose: "prefers an atomic same-filesystem `rename(2)` of the entire directory … never overwrites anything that already exists in the new directory, and never deletes from the legacy directory". XDG is also a packaging contract: "user data under XDG_DATA_HOME is never package-owned" — uninstall/purge must not delete user data, and release CI verifies exactly that.

9.2 Configuration: env vars only, no CLI

There are no command-line arguments anywhere (verified by enumerating every `env::var` read); precedence is `SPP_*` overrides → the settings table (read at boot; `log_level` is live-reloaded through a tracing reload layer, clean-audit item 138) → defaults in `config.rs`. The old `AppConfig` struct "carried four fields nothing consumed; only `data_dir` was ever read" (clean-audit item 153) — dead config surfaces are deleted, not maintained. Credential resolution order: environment first (CI, power users), then the encrypted envelope written by the in-app settings form, "so the app connects without any environment variables."

9.3 Single instance: flock, not pidfiles

`main` acquires an exclusive non-blocking `flock(2)` on `instance.lock` *before* the DB opens; a second instance prints a clear message, logs, and exits 1 (`single_instance.rs`). Two processes writing one SQLite file is the disaster this prevents. The Tauri plugin's "focus the existing window" behavior was consciously *not* reimplemented: doing it right needs IPC between two native processes, which does not exist yet — the comment says exactly that, making it a recorded scope cut rather than an oversight. The crate's only `unsafe` is the two `libc::flock` FFI calls;

the kernel releases the lock on crash, which is why there is no staleness problem to solve.

9.4 The no-simulation law, enforced

Since 1.2.0 there are exactly two boot modes, and "*nothing is ever faked*": the pre-1.2.0 simulated demo account "produced bug reports about tickets that didn't exist and trained users to distrust the mirror", so it was deleted — dataset, boot path, and settings rows (m050). Two guards keep it dead: `scripts/gate_no_simulation.py` runs first in CI and in the release build, and the watermark-poisoning repair (v1.4.0, R2/R3) forces `sync_state` back to `NEW` when real credentials appear after an empty session — otherwise a stale watermark from an empty run would skip every future sync. Any new boot-time provider state must respect that transition policy.

CHAPTER 10 · PART III

The sandbox

Landlock plus seccomp around untrusted document parsing — and why EPERM, not a kill.

The only code that touches fully attacker-controlled bytes is PDF/DOCX/ZIP parsing in the Knowledge importer. The module doc (`core/src/sandbox.rs:1-9`) states the threat model in one sentence: "A memory-safety bug that turns into code execution must not be able to read user data, reach the network, or persist anything." The answer is a per-thread sandbox: **Landlock** restricts filesystem access to nothing (a ruleset handling all path rights with zero rules), and a **seccomp** filter allows only the compute/syscall baseline, returning `EPERM` for everything else — socket, open, execve, clone, ptrace stay blocked.

10.1 Why EPERM instead of killing the thread

Filters stack but can never be replaced, so the allow-list cannot be loosened afterwards; and the denial action is deliberate: "a killed thread cannot notify its channel (the caller would hang until the timeout), while EPERM still denies every escape path with a clean error" (`sandbox.rs:16-22`). A 120-second task timeout bounds a wedged parse, with the note that "input sizes are already capped upstream … so a parse that exceeds this is either wedged or the caps need revisiting."

10.2 Descriptor hygiene (audit fix M13) and the TOCTOU answer

Landlock cannot govern already-open descriptors, and the exploited parser inherits the host's fd table — the mirror DB, `secrets.key`, the listener sockets. Hence three rules: `read` is pinned to the process's own stdio (every sensitive descriptor

sits at fd ≥ 3, and `open` is not allowed anyway, so the parser can never grow the set); `mmap` is permitted only for anonymous (file-less) mappings; and `close` is denied outright — the sandbox owns no descriptors, so "every close it could attempt targets a host fd". The time-of-check-to-time-of-use problem is avoided *structurally*: the closure receives already-buffered bytes ("everything it needs has to be moved in"), so the sandboxed code never resolves a path at all. Outside the sandbox, the knowledge-import jail closes the path race the ordinary way — lexical `..`-collapse, then `canonicalize`, then prefix re-check against `data_dir/knowledge-import` — with size caps applied *before* allocation and ZIP entries bounded by `take()` "because zip declarations can lie".

10.3 Why a Cargo feature, and why packaging forces it

The `sandbox` feature is off by default so `cargo test` / `cargo run` work unchanged on any OS, and on because shipped artifacts must carry it: `package.rs:161-181` passes `--features supportos-plus-core/sandbox` explicitly (audit item 40). If `apply_restrictions` errors, "the parse refuses to run unsandboxed rather than silently losing the hardening"; on kernels without Landlock, that layer degrades with a logged note while seccomp still applies. Containerization was rejected as the alternative: the product is an end-user desktop app, and per-parse thread lockdown is strictly finer-grained — the parser cannot see the DB even though the process legitimately can.

Sharp edges

- Build with plain `cargo build --release` and you ship **without** the sandbox — always package via `cargo xtask package`.
- Route any new attacker-controlled parse (e.g. `.eml`) through `run_sandboxed` too.
- The regression tests assert `EPERM`, not `EBADF`, to prove the filter fires before the fd lookup — do not "fix" them.

CHAPTER 11 · PART III

Secrets and the envelope

One AES-256-GCM envelope, one master key, and the rule that plaintext never enters the database.

Every secret class in the app — the Help Scout webhook secret, the OAuth access/refresh tokens, the Docs API key, every BYOK cloud AI key — is stored the same way. `settings.rs:8-16`: "*`secrets.value` stores*

`0x01 || nonce(12) || AES-256-GCM ciphertext` sealed under a per-installation master key (`secrets.key`, created with 0600 permissions on first boot). Plaintext never enters the table. This replaces the previous fiction of a 'secrets' table that nothing ever wrote and that the schema comment claimed was encrypted." The crypto lives once in `crypto.rs` because "only the envelope layout differs between formats, so the primitives live here."

11.1 Why not the OS keyring

The envelope must survive headless CI smoke boots, in-memory test databases, `SPP_DATA_DIR` relocation, and the one-time legacy data-dir migration that moves the DB and the key together. No keyring daemon is guaranteed on minimal Linux desktops, and keyring rotation would silently destroy the mirror's credentials. The threat being addressed is honest and limited: other local users and casual file copying — the right level for a single-user desktop app.

11.2 Key lifecycle rules

- `write_key_file` **refuses to clobber** an existing key: "keys are created once and must never silently rotate, or every stored secret would become undecryptable."
- Losing `secrets.key` makes every stored secret unrecoverable *by design*; a malformed key file is refused, not replaced.
- A corrupt envelope or a ciphertext failing GCM authentication "surfaces as an error rather than being silently treated as 'missing'" — the audit repeatedly converted silent emptiness into honest errors.
- The migrations m049 (webhook secret) and m053 (OAuth tokens out of plaintext columns; the row "keeps metadata only — its token columns are permanently empty") moved existing secrets into the envelope.

11.3 Secrets never appear in logs, Debug impls, or the repo

Credential structs carry manually redacted `Debug` impls ("the derived `Debug` carried every credential verbatim" was the fixed bug); redaction is char-boundary safe after audit item 107 (a byte-offset slice once panicked on multi-byte client ids). A repo-wide hygiene test in xtask fails CI if any file named `secrets.key` reappears — the forward guard from the history rewrite (Chapter 29). One residual risk is recorded in `repo-audit-2026-10-24/FINDINGS.md` (FIND-003): QA evidence directories once contained captured key files; the history was rewritten and the finding kept as a record — know it, do not repeat it.

Backups and restore

VACUUM INTO, 0600 everywhere, verify before swap, and the atomic move-aside that closes the stale-WAL window.

Backups are pull-based export files: `VACUUM INTO` produces a crash-consistent snapshot (including WAL content) in one call, immediately chmod'd 0600 by `crypto::restrict_permissions` — audit fix M12, because "backup copies, plaintext snapshots/temps and JSON/CSV exports used to inherit the umask (0644 on default systems) and were world-readable on multi-user machines". The chmod is best-effort by design: an export that dies because chmod failed on a foreign filesystem is worse than a 0644 file the user just exported. A redacted settings sidecar (`.settings.json`) accompanies each backup; audit fix M18 narrows how its path is derived — "*only the file EXTENSION may change here*", after a `replace(".db", …)` once rewrote every ".db" occurrence in a data dir path and produced sidecars inside nonexistent directories.

12.1 Verify first, touch nothing before

`backup_service.rs:216-219` (audit item 11): "*never swap an unverified file over the live database. The integrity check catches truncated copies and non-databases; the structural probe catches foreign SQLite files. Both run BEFORE anything on disk is modified.*" The structural probe exists because a foreign-but-valid SQLite file "would pass a bare integrity check yet brick the app once swapped in" — a SupportOS database always carries the base-schema `app_state` table. Refusal messages promise the invariant: "The live database was not touched."

12.2 The swap, and the window it closes

The restore ordering is the reference implementation for every DB swap in the codebase (the `.sosync` importer cites it, audit fix H7): stage the verified copy and fsync it; **move the live database and its WAL/SHM aside as one unit**; rename the new file in; fsync the file and the directory entry. The comment explains the stakes: the previous code renamed the new database into place first and only then deleted the old `-wal`, "so a crash in between would have paired the fresh file with the old WAL and replayed foreign pages onto it. With the live side moved first, between the renames the db path simply does not exist — SQLite treats that as 'open fresh', never as a cross-generation pair." Crash-window consequences are documented: mid-swap death leaves the previous generation on disk as `<db>.restore-prev` for

manual recovery — "fresh-empty beats foreign-WAL replay". Restart-to-activate is part of the contract, because open connections keep pointing at the old inode.

12.3 Replace, never merge

Restore replaces the whole database. The mirror is a *derived copy* of Help Scout state; a merge would need per-table conflict rules, could resurrect deleted rows, and could pair new rows with a stale sync cursor. Schema compatibility is a fingerprint comparison that deliberately excludes `_migrations` and FTS shadow tables; a compatible ancestor schema is accepted and migrated up on restart; anything else is refused before disk mutation, after writing a `.pre-import-*.db` safety backup. The workspace reset complements this: it "walks the live schema instead of a hand-maintained delete list: every table not on the KEEP allowlist is emptied. A test pins the allowlist against the migrated schema, so a name typo fails the build, and a table added by a future migration lands in the purge set by default — its author must make a conscious KEEP decision" (`data_reset.rs:8-13`).

> **Rejected: continuous WAL archiving**
>
> Point-in-time replay machinery would add an archiver process, a recovery tool, and three new failure modes — for data a re-sync from Help Scout can mostly regenerate anyway. The user-facing message keeps the PII risk visible: "this export contains customer data - handle it carefully."

CHAPTER 13 · PART III

Bundles, resets, secure delete

The encrypted cross-device channel, and what "delete all workspace data" actually promises.

13.1 The .sosync bundle

The cross-device/offsite channel is a scrypt($N=2^{15}$)+AES-256-GCM bundle whose passphrase check *is* the integrity check: AES-GCM authentication makes wrong-passphrase and tampered-bundle indistinguishable, and both are refusals before anything touches the DB. Three hardening layers carry audit markers: M15 bounds the header-declared scrypt N to [2^{14}, 2^{25}], power-of-two only — "a crafted bundle with $N = 2^{31}$ forced a multi-GB scrypt allocation (local DoS) before the passphrase was ever tested"; clamping was rejected because "clamping would silently derive a wrong key and surface as a misleading 'wrong passphrase'". M11 is a scope guard for the plaintext export snapshot: "the pre-fix code only removed it on the happy path, so any error after VACUUM INTO — or a panic — leaked plaintext customer data

into the bundles dir". Clean-audit item 48 sweeps stale `.import-*.tmp` files (which hold decrypted plaintext) with age thresholds — never "delete all temps", or you kill concurrent operations.

13.2 What secure delete means here

The reset (Settings → two-step "delete all workspace data") turns `secure_delete = ON` for the purge, then runs a separate `VACUUM` (audit fix M14): "plain DELETEs leave the purged rows recoverable in the file's free pages and WAL. secure_delete = ON makes SQLite zero every freed cell/page at delete time … Best-effort by design: if the pragma errors, the reset must still proceed (warned) — a wipe must not be blockable by a pragma." The VACUUM is separate so a failed compact cannot un-report a successful delete. The pragma is toggled per-operation, not globally, "because zeroing every delete forever would tax the hot write path for a benefit only the reset needs." The code acknowledges its own limits honestly: this is page/WAL zeroing, not media-level erasure — and a 64-pass dangling-row convergence loop inside the FK-suspended bulk delete makes "the commit either fully consistent or the whole reset rolls back."

CHAPTER 14 · PART III

Webhooks and OAuth

The loopback listener's two routes, HMAC discipline, dedup with leases, and why there is no PKCE.

The only inbound HTTP is a `127.0.0.1`-only listener serving exactly two routes: `POST /webhook` (Help Scout event ingress) and `GET /oauth/callback`. It speaks just enough HTTP/1.1 with strict caps (16 KiB headers, 1 MiB body, 10-second read timeout) "so a stray local process cannot balloon memory". If port 3000 is taken, the app degrades to log-only warning plus a UI badge (external audit FIND-001) and continues — webhooks are load-bearing only for Beacon chats.

14.1 Webhook discipline

- **Unsigned = hard 403, always** (migration-map row 1): "accepting them would let anyone with loopback access inject fake Help Scout events." HMAC-SHA1/base64 with constant-time tag comparison.

- **One secret-resolution function** (audit item 32): registration once capped secrets at 40 chars while verification used the full value — "every delivery failed the check, ten discards deactivated the hook, and the health check re-registered it into the same trap. Both sides now resolve through this one function."
- **Dedup by content hash** (`event_hash UNIQUE`) because Help Scout retries on non-2xx; audit fix M8: a duplicate "stamped the ORIGINAL row 'duplicate' — overwriting 'pending' … and declaring the event handled while Help Scout was still retrying it. The original's processing state is left untouched; the delivery simply answers as a duplicate" (200 with `duplicate: true` , so the platform stops retrying without double-processing).
- **Lease recovery + state-aware prune**: rows stuck in 'processing' past a 15-minute lease return to pending (5-attempt budget); the prune (clean-audit item 94) only removes terminally-done rows — the pre-fix prune "kept the newest rows REGARDLESS of state … so the prune deleted work that was never processed" during a backlog.

14.2 OAuth: authorization code, no PKCE, and why

The browser flow is plain authorization-code with a single-use state: 10-minute TTL (clean-audit item 93 — "an unconsumed state issued weeks ago must not remain redeemable"), consumed by a **single UPDATE whose predicate carries the full unconsumed+fresh condition** (the audit note records that SELECT-then-UPDATE was a TOCTOU), and bound to the exact loopback port recorded at issue time — "a state recorded for a different port … is rejected before the code is exchanged". PKCE was deliberately *not* implemented: its protection targets authorization-code interception on an untrusted channel, while here the browser and the app share one host and the client secret lives in the envelope; the Settings UI recommends client credentials anyway ("no browser, no redirect URL"). The tokens never touch plaintext columns (m053); revocation "must not leave readable material behind". The listener even fingerprints itself so "a foreign program squatting on the port (a dev server on 3000 is the classic case) is caught too."

Untrusted content

Customer HTML, user-configured URLs, imported files — the three ways the outside world reaches the render path.

15.1 HTML: allowlist plus CSS-obfuscation decoding

Conversation bodies arrive as arbitrary HTML. `sanitize_thread_html` enforces a fixed tag/attribute allowlist (schemes: http, https, mailto, tel, data, cid; `rel="noopener noreferrer nofollow"`; protocol-relative URLs denied; `data:text/html` images stripped twice). The interesting part is clean-audit item 50: "the blocklist must see through CSS-level obfuscation. It used to run on the raw attribute value, so a comment inside a keyword (`ur/*x*/l(`) or a backslash escape (`\75 rl(`) sailed past it while the CSS engine still read `url(` . The list now runs on the decoded form" — decoded to a fixpoint, with a 64-pass bound and a 2000-char clip that are a matched pair (change one without the other and layered-escape bypasses reopen; tests pin the deep-layering rejection). The richtext renderer's `rm_tags` guard (item 63) pins an ammonia API subtlety: it removes tags *from the allowlist*, not adds.

15.2 URLs: the SSRF guard, sync and async twins

Connectors fetch arbitrary user-entered URLs on a machine holding the DB and the master key. `security.rs` rejects the full private/loopback/link-local set with named reasons (including 169.254.0.0/16 " — cloud metadata"), numeric/hex address tricks, and `.localhost/.local/.internal` names; then every DNS-resolved address is re-run through the same check, closing the DNS-rebinding-shaped gap where a public name points inside. Clean-audit item 51 exists because the sync guard "blocks the thread on getaddrinfo — correct on worker threads, catastrophic on an async executor (a slow resolver stalls every task on that worker): async callers must use this variant." Redirects are `Policy::none()` here and in the Help Scout clients — see 15.3. The accepted residual risk is recorded: the guard resolves DNS once; a rebinding server can rotate between check and fetch.

15.3 Redirects are data, not transport (M5)

The Help Scout client does not follow redirects, for two independent reasons. Correctness: Help Scout's 301 "conversation merged" answer carries the target id — "the client used to follow redirects, which turned that into a silent 200 for the TARGET conversation — merged originals were re-ingested as their target and

stayed stale forever." Security: never surfacing a 3xx removes the token-leak class entirely. The 301 handler converts the Location header into a typed "Conversation merged into {id}" outcome that the sync engine marks honestly. Related platform quirks are recorded in the same module: 201 replies carry the new id in the `Resource-Id` header ("callers do not read a phantom 0"), and the Docs API wraps every list response in a keyed envelope — the pre-fix code "read the top-level items, matched nothing and every pass silently synced 0 collections" (audit C2).

Sharp edges

- Any new HTTP client must set its redirect policy explicitly and run the SSRF guard (async variant on async paths).
- Any new write of customer data calls `crypto::restrict_permissions` immediately.
- POSTs are never auto-retried after ambiguous failures — or a customer receives the same reply twice.

CHAPTER 16 · PART IV

Local-first AI

One built-in provider (LM Studio), a default of None, BYOK cloud as generic rows — and the kill-switch hierarchy.

The provider-kind enum has exactly two values: `None` ("None (no AI provider)") and `LmStudio`. Fresh installs ship with no AI at all, and `ai_features_enabled` requires *both* an explicit provider and a chat model. The rationale is the product's privacy spine: support transcripts never leave the machine, everything works offline, marginal cost is zero, and — the subtle one — **the AI is the only non-deterministic component, so everything around it (evidence packs, guards, prompts, evals) can be deterministic**. The module `ai_local.rs` records the problem the default solves: "with nothing configured the AI features looked completely dead."

16.1 Endpoints, not providers

There is no per-vendor SDK. The endpoint list (`ai_hub.rs`) is seeded on first use with two rows — LM Studio (local, enabled, inheriting any pre-hub base URL) and Ollama (local, optional) — plus any number of user-added OpenAI-compatible cloud rows. "Provider" reduces to *endpoint row + optional key + per-Mtok prices*. BYOK (bring your own key) is decision D-a: opt-in per feature, keys stored only in the encrypted envelope, the stored JSON referencing them by key name. Probes authenticate "exactly like chat/embed [23] keyed (BYOK cloud) endpoints 401 the

unauthenticated GET and showed 'offline' while the same endpoint served real calls." The demo `FakeAiProvider` / `NoopAiProvider` doubles were purged in 1.4.0 (R7/D14): "their fake-response strings were exactly the simulated-data residue this purge removes."

16.2 Two-level gating: kill switch + 16 features

A global `ai_enabled` switch dominates 16 named feature gates (`AI_FEATURES`). The default policy is stated where the list lives: "every new AI surface is registered so the kill switch and the AI Center toggle grid cover it. **On-demand surfaces default ON (copilot precedent); automatic generative ones stay OFF.**" The mapping encodes risk: automatic generation writes content the user did not ask for; on-demand surfaces have a human trigger per run. Gates are evaluated at *execution* time — a queued job whose feature was toggled off meanwhile is "skipped honestly". The kill-switch click is arm-confirmed (FIND-005: "the write used to run on the click — arm the shell confirm instead, one click disabled all AI behavior"). Unknown features fail closed (`ai_feature_enabled` returns false), never open.

Rejected

First-class OpenAI/Anthropic provider kinds (vendor SDKs to maintain, telemetry pressure, bundled keys); a dedicated `ProviderKind::Ollama` (Ollama speaks the same OpenAI-compatible protocol — a second code path buys nothing); keys in plain settings rows (the envelope exists); per-feature toggles without a master switch (no single emergency off).

Sharp edge

Two config layers coexist by design: the role hub (preferred) and the legacy `ai_settings` / `lmstudio_*` row (the pre-hub fallback the AI Center provider panel writes, item 151). `backend_for_role` wins when a role is assigned; don't "simplify" one layer away without tracing both precedence orders.

CHAPTER 17 · PART IV

Five roles and the golden set

Why every inference is tagged Embed/Classify/Summarize/Draft/Agent, and why changing the map re-runs nine canned cases.

Every model call carries one of five roles (`ai_hub.rs:29-31`). The design point is that a local GPU runs a *portfolio*: Classify ("triage, sentiment, topic, urgency — high volume, low stakes") wants a tiny fast model; Draft ("customer-facing drafts — the

only role where cloud is recommended, still opt-in") wants the biggest one; Embed, Summarize and Agent have their own economics. Per-role mapping also scopes blast radius: a bad Draft model degrades drafts, not triage. The rejected alternative — one global chat model — survives only as the documented legacy fallback for pre-hub installs.

17.1 Commit semantics: once per edit, never per keystroke (item 74)

`commit_role_map` persists a role-map edit and re-runs the golden set **once per committed edit**, on field blur, and only when the live map differs from the last saved snapshot. The comment explains the failure it fixed: "typing no longer persists and enqueues a golden_eval per character." `RoleMap`'s `PartialEq` exists precisely "to power the AI Center's persisted-vs-live comparison."

17.2 The golden eval: a scoreboard, not a gate

Nine seeded cases (`ai_eval.rs`, defined in `ai_pipeline.rs:315-326`) exercise the real analysis prompt end to end: trivial, compound, ambiguous ("It does not work"), known-issue (DST), history follow-up, timezone (Santiago), integration (Slack), billing (declined card), and urgent escalation. A case passes when the model returns a parseable analysis with at least one non-empty core field (summary/intent/customer_goal). Triggers: on demand, on role-map commit, and via a SHA-256 fingerprint over (prompt versions + role map) that the daily maintenance pass checks for drift. Deliberate choices to preserve: **eval failure never blocks saving** the role change — the routing change stands and an honest failed row is recorded ("a disabled backend still writes an honest run row — passed=0, failed=N"); and the evals live as runtime jobs, not `cargo test`, because they need a live model. Know the limit: the eval measures *parseability + core-field presence*, not factual accuracy — a model can pass while being wrong.

CHAPTER 18 · PART IV

Guardrails

Evidence packs, VERIFIED ANSWER MODE, a second verification pass, and the memory quarantine — the defense-in-depth around a model you cannot trust.

The stack's premise: a local model *will* hallucinate; the defense is architectural, not prompt-politeness. Five layers:

1. **The model only sees a bounded pack.** `build_evidence` assembles threads (1500-char excerpts), last-5 customer history, similar cases (FTS + semantic), 3 known issues, 4 knowledge hits, 3 saved replies, and quarantined-filtered memory facts. Every section carries visibility and maps to persisted source rows.
2. **Internal-only material is stripped in code, not asked away.** The draft renderer enforces `retain(visibility == "customer_safe")` on similar cases and knowledge — the prompt says it too, but code is the mechanism.
3. **The draft prompt is VERIFIED ANSWER MODE:** "NEVER invent: timeframes, feature availability, bug-fix status, release dates … If the evidence does not support an answer, say that verification with the team is needed." The draft's contract is a JSON envelope (`{"draft": …, "used_evidence": […]}`) — item 39 records what happened when a prose answer broke it: the customer-facing draft *became the raw envelope*; now "a parsed envelope without a usable draft fails loudly."
4. **A second, differently-prompted pass verifies.** "You verify AI-drafted customer replies against evidence. You are strict and skeptical." Parse failure of the verifier itself yields `verified: false` — fail-closed, "treat as unverified", never silently blessed.
5. **Provenance persists.** Every draft stores run id, prompt version, and its source rows; the UI shows verification outcome next to drafts.

18.1 The memory quarantine: deterministic by design

`ai_memory_guard.rs` quarantines personality/mental-health vocabulary (16 words + 8 prefixes + case-sensitive whole-word IQ/EQ) from every prompt and tool result by pure string logic. Why not a classifier model? Non-deterministic, slow, itself jailbreakable — and the threat is the AI drifting into psychoanalysis of customers ("difficult personality", "anxiety" labels that poison every future interaction). Why not block memory extraction? It would kill a useful feature. The accepted misses are recorded: paraphrases outside the vocabulary, non-English equivalents. This is a *memory-content* policy, not a prompt-injection defense; PII crossing the model boundary is a different layer (`redact_text` , item 97 — "the rewrite lanes carry agent-typed draft text — quotes of customer-supplied card numbers, secrets and keys included — across the model boundary").

The read-only agent

22 tools, zero writes, a hard budget — and why "write tools with approvals" was refused.

The copilot's tool surface is `CopilotTool::ALL` : 22 read-only query tools over the local mirror. Enforcement is **structural**: `ai_tools::execute` is a `match` over read queries only, and the fallthrough is the policy statement: "*Unknown tool {name} - allowed tools are read-only search tools.*" There is no INSERT/UPDATE/DELETE in the executor; output is redacted on the way out. The system prompt binds the rest: "You have READ-ONLY tools … ANSWER WITH EVIDENCE … Never fabricate ticket numbers, dates, resolutions or quotes … You do not send anything, change any ticket, or write to Help Scout. You only read and explain."

19.1 The budget, and the protocol bug it survived

Five tool rounds, eight calls, 4000-character results — caps on prompt bloat and runaway loops. Clean-audit item 44 explains why every over-budget call still needs a response: "every assistant tool_call must carry a matching tool response — strict OpenAI-compatible servers reject the follow-up request with a 400 when a tool_call id dangles. The un-executed calls now get an explicit budget-refusal result instead of silence," followed by a forced final answer: "Answer now from the evidence you already have, and say plainly which parts you could not verify."

Rejected: write tools behind approval prompts

The approval UX already exists (Automation); duplicating it for an agent "invites automation-of-destruction." The worst case of a hallucinated read call is a wrong answer; the worst case of a hallucinated write is a mutated ticket or a sent email. If you ever add a tool, it must remain a SELECT over the mirror, clamped and redacted, added to `CopilotTool::ALL` and the AI-visibility rules — never a write path.

Vectors and hybrid search

sqlite-vec behind a reversible trait, RRF fusion with honest fallbacks, and why you never change chunk geometry casually.

Dense vectors live in a sidecar SQLite file (`vectors/vectors.db`) as a single `vec0` virtual table with cosine distance. The decision (D4) was made by a *gate*, not a preference: "Gate: `xtask vector-count` on the live DB. <100k chunks → GO (dense vec0, sparse FTS5, RRF untouched, exact pin). ≥100k → qdrant-edge stays … `VectorStore` trait keeps it reversible." The embedded qdrant engine was removed when the count passed the gate — one less engine, backups/restore/reset unchanged — but the trait seam means the store can be swapped back without touching call sites.

20.1 Dimension is destiny

`vec0` tables need the dimension fixed at `CREATE VIRTUAL TABLE` time; the collection is rebuilt when it changes ("Dimension change = model change: rebuild the collection — chunks re-embed through the normal pending pipeline"), and a mismatched upsert fails loudly. Two facts a newcomer must internalize: **embeddings are NOT normalized** by this codebase (correct under cosine, which is scale-invariant; score = 1 − distance; any dot-product math you add must normalize first), and the only `unsafe` in core is the sqlite-vec FFI registration.

20.2 Point ids, orphans, and batch integrity

Clean-audit item 21 is the war story that explains `POINT_ID_STRIDE`: "knowledge chunk 7 and conversation chunk 7 used to land on the same `point_id`, so the upsert's DELETE+INSERT silently overwrote whichever family had written first." Point ids are now family-banded, and a cross-family overwrite "fails loudly instead of colliding." Item 60's orphan sweep removes points whose chunks were deleted or re-chunked ("a re-chunk hands every chunk a NEW id, so the old points kept matching with their stale text snapshots forever"). Item 46 partitioned embed batches: "One empty vector poisoned the batch: a zero-dim point landed in the store and its chunk was marked done, never retried. The batch is now partitioned per chunk."

20.3 Hybrid retrieval: RRF with printed excuses

Search is BM25/FTS5 first; when the retrieval feature is on, an Embed role is routed, and the index is available, the query is embedded and dense hits are fused

per lane via Reciprocal Rank Fusion, k=60 — "the classic RRF rank constant, pinned by the search tests." Rank-based fusion means the two lanes' incompatible score scales never need calibrating (a dead fusion stack once got deleted precisely because it fused unjoinable id spaces, item 154). Lane caps (tickets 40, knowledge 10) mirror page caps; FTS-only lanes are rescaled onto the RRF scale when fusion ran, "so the cross-lane ordering is meaningful." Every degradation prints *why* — the four mode notes are verbatim in `search.rs:819-844` ("FTS only — the vector index is off (Settings > Vector index)." etc.) — and `used_semantic` is set only when vector hits actually made the list.

20.4 Chunking economics

1200 UTF-16 units with 150 overlap, preferring sentence boundaries; thread bodies pre-truncated to 4000 chars each; resolved conversations get one labeled `resolution` chunk — "the last published reply before closure, labeled so retrieval can prefer it for 'how was this solved' queries. Nothing fabricated — an unresolved or empty thread simply gets no resolution chunk." Each embed pass takes at most `EMBED_PASS_LIMIT = 60` pending chunks per family (keep worker ticks short; a large backfill drains 60 at a time), and failures retry under an attempts cap of 5 (item 100: "a failing chunk was retried on every pass without ever approaching a cap").

Sharp edges

- **Never change 1200/150 casually** — there is no chunk_version column anymore; a geometry change with old vectors in the store silently mixes embeddings. Plan a full re-embed.
- Never upsert vectors with raw per-table ids; use the family-band namespacing.
- New embedding model ⇒ `ensure_collection(new_dim)` rebuild + pending-pipeline re-embed, never an in-place dimension edit.

CHAPTER 21 · PART IV

The ledger and the prompts

Every run is a row; every prompt is a versioned constant; every parameter is fixed in source.

Every inference writes an `ai_runs` row: tokens, latency, model, endpoint, estimated cost, prompt version, and the response body. The table was "born as a cache, grown into the ledger" (m008 indexed `(input_hash, prompt_version, model)`) — with BYOK cloud in the picture, cost is user money and spend must be observable per

feature per day before it hurts. The contract: every `start_run` ends in `complete_run_with_usage` or `fail_run` — "the ai_runs row stayed 'running' forever and the AI Center rendered it with a green badge" was the fixed bug. Clean-audit item 43 extended the same honesty to the copilot: "every copilot chat was invisible to the cost ledger." Requests are hashed, never stored; responses are stored (that is the replay/debug story). Retention defaults to **off** — know that `ai_runs` grows unbounded unless the user sets a retention window.

21.1 Prompts are code, and version bumps are load-bearing

All system prompts are versioned constants in `ai_prompts.rs` (13 of them). They are code-adjacent contracts: the draft renderer's visibility filter, the JSON shapes the parsers expect, the [^N] citation markers the UI renders. DB-editable templates were removed, not built — item 149 deleted the dead prompt surfaces: "an edit would silently break parsers, void eval comparability, and bypass review." Bumping a version does three things: misses every cached analysis (new prompt_version ⇒ cache miss ⇒ honest re-analysis), flips the golden fingerprint (the maintenance pass re-runs the nine cases), and keeps the ledger attributable per day. Old `ai_runs` rows keep their version strings — never "clean up" old versions; the cache index needs them.

21.2 Generation parameters are fixed per lane

LANE	TEMP / MAX TOKENS	WHY
Analysis, drafts, verification, memory, attributes, golden eval	0.15 / 2048 (draft 1600), json_mode	Near-greedy + parseable; JSON is the downstream contract
Copilot tool chat	0.2 / 1600, no json	Conversational but must quote evidence
Rewrites (Shorten/Expand/Warmer)	0.3 / 1600, no json	Style freedom; prompt forbids new facts; human reviews before send
Translation, QA, coaching AI	0.1 / 2048·800·700, json where parsed	Strictest lanes; fabrication-intolerant

Fixed in source, not user-editable, "because params are load-bearing for the golden eval and cache identity — a user-tweakable temperature would make eval scores incomparable." The 120-second HTTP timeout absorbs cold model load + long prompts; `stream: false` always (SSE was dropped with the old stack; non-streaming makes token accounting exact). Item 101's parse strictness is the ledger's last line of defense: "a well-formed JSON 200 whose choices are empty … used to surface as a SUCCESSFUL completion whose content was the literal string

'Unknown' — a response with no readable content is a labeled error, never fabricated text."

CHAPTER 22 · PART V

Automation

Two kill switches, an approval matrix sized by blast radius, and a once-guard with a memory.

Automation is off. Twice. The engine gates on `automation_enabled` (default `false`) and the write path gates again on `automation_write_actions_enabled` (also default `false`). The philosophy is stated in the workload module's user-facing note: "Suggested assignee is a read-only recommendation. SupportOS never reassigns automatically unless you explicitly enable an approved automation." The failure mode being avoided is not hypothetical — the once-guard story below is an auto-ack storm that actually happened.

22.1 The approval matrix is a blast-radius table

`automation.rs:83-92`: Assign and ChangeStatus always require approval; SetPriority requires it only when the new priority is `"urgent"`; AddTag and SendNote never do. The mapping asks two questions — how much state can the action corrupt, and how visible is the mistake in a busy inbox. The idea scales to the 9-action engine as risk tiers (`read` / `non_destructive` / `higher_risk`), where every higher-risk action goes to approval unconditionally, "with the log string 'Action {kind} is a write action and requires explicit approval'". Per-rule free-form risk settings were rejected: "a solo maintainer cannot audit per-rule choices; the tiering is code-owned and uniform." Sharp edge: `SetPriority { "high" }` does *not* require approval — the string compare is exactly `"urgent"`.

22.2 The once-guard (m060, clean-audit item 57)

The only event source is the sync mirror, which re-delivers "conversation changed" on every pull. The pre-fix code "fired BOTH the `new_conversation` and `customer_reply` triggers on every customer-latest sync (a reply re-sent the 'new conversation' auto-ack)". The fix gave `conversations` a memory column — the newest customer thread id whose triggers the conversation has already consumed — with a decision table: first customer thread ever → `new_conversation`; a later thread → `customer_reply`; no new thread (tag change, status flip, re-sync) → neither. "Record the thread as consumed only when the rules actually ran; a failed firing … stays unrecorded so the next change retries it." This converts an at-least-once

event stream into exactly-once-per-thread semantics without a queue. Any new fire path must replicate the consumption discipline.

22.3 Outcomes are facts; there is exactly one retry

Every run inserts an `automation_runs` row (`completed` / `failed` / `awaiting_approval` / `skipped` with human-readable detail). The one retry that exists is deliberate and loud (audit item 19): "an approved write action must reach Help Scout. The typed local write alone was silently reverted by the next sync — the manual inbox path queues an outbound push after the same local write, and this is that queue entry … A failed enqueue propagates so the approval job retries instead of reporting a silent half-action." In a mirror-synced app, "local write succeeded, platform reverted it" is the deadliest bug class; at-least-a-record plus visible retries is the answer. Blind in-engine retries were rejected because automation mutates Help Scout — the outreach module's "infinite retry storm" (Chapter 25) independently proves the danger. No cron triggers exist either: a timer layer would need wake-the-app semantics and catch-up machinery, and every useful scheduled concept is computed at sync/sweep time against fresh mirror data. Sharp edge: the `Trigger::SlaRisk` variant is selectable in the rule form but nothing currently emits the event — it is a dormant path (Chapter 32), not a working trigger.

CHAPTER 23 · PART V

SLA and workload

Determinism as a feature: numbers a human can audit, notes that travel with the number.

SLA defaults (24 h first response, 72 h resolution) are schema-level; per-mailbox overrides win; "mailboxes without business hours or targets are listed as unconfigured — nothing is guessed" (`ALERTS_NOTE`, rendered verbatim in the UI). At-risk is `AT_RISK_RATIO = 0.8`: one warning beat before breach "without crying wolf at 50%". Business minutes are used because wall-clock SLA lies across nights/weekends; the alerts cap of 50 truncates worst-first after true totals are counted — "the 50 most actionable rows" for a desktop table. Three audit markers define the honest clock: item 13 (the waiting clock is "the newest published customer thread", not `updated_at`, "which flips on every status change, tag edit, assignment or sync touch, which silently reset the waiting clock"), item 53 (at-risk "judges each conversation against the target it can actually miss" — replied ⇒ resolution, else first-response), and item 141 (a bad business-hours `days` list "passed the raw is_empty

check and then filtered to nothing downstream — the window silently DISABLED"; it now degrades to the default schedule, never disables).

23.1 Workload: a formula you can defend in a standup

The workload model is deliberately linear: pressure = weighted open load / capacity, with weights urgent 3.0, sla 2.0, waiting 1.5, open 1.0 and a default capacity of 25 clamped to 1–500. ML was rejected for reasons the module writes down implicitly: training data does not exist on a single-machine product, and "deterministic, explainable, testable" is the entire point — a manager must be able to ask why the suggestion was made and get an answer made of arithmetic. The module also documents its own epistemic honesty: "assignment changes mid-conversation are not historically reconstructable"; availability is "shown as unknown, never guessed." Dead code was removed with a tombstone: `AgentWorkload` / `TeamWorkload` "used to live here — dead since introduction (zero call sites) and carrying a latent id-domain bug (`assignee_local_id = ?1` fed a *remote* id). Removed rather than fixed so the latent trap can't be picked up by future callers." The local-vs-remote id trap is the single most recurring bug class in the codebase — grep before you assume.

CHAPTER 24 · PART V

Reports and segments

A closed registry with printed excuses, a builder that cannot fabricate, and segments that are trees — never SQL.

Reports is a fixed registry of 32 Help Scout Reports API endpoints (a test enforces unique keys/paths; the count is reconciled in-code from the docs' 33) plus a local metric×dimension builder over the mirror. The registry's module doc is the design contract: "capability detection so Standard plans get one honest sentence instead of empty charts … **Nothing here fabricates report data: the cache is written only from real provider responses, and the capability verdict drives every UI claim.**" Only 403 and 2xx are verdicts; 401/network stay Unknown; the verdict resets when credentials change. The builder's 21 metrics each carry `definition` and `limitations` strings that the run echoes back as notes — survivorship bias, unmeasured pre-sync history — "the honesty layer." Everything is computed in Rust over the mirror, which is what makes reports work offline and on plans without the Reports API. Free-form SQL and CSV export were both rejected: the first "breaks the nothing-is-fabricated contract, invites full-table scans on a desktop SQLite file, and has no capability story"; the second because drill-down happens via `sample_conversation_ids` in-app instead. The only CSV in the product is the Settings

conversations export, which carries its own formula-injection guard (clean-audit item 106: a subject like `=HYPERLINK(…)` "became a live formula").

24.1 Segments: the tree is the contract

Segments are typed condition trees — 13 node kinds, `MAX_TREE_DEPTH 10`, `MAX_NODE_COUNT 200`, 50 conditions per list — evaluated to customer-id sets in Rust, never string-built SQL. The tree buys four things: save-time validation, a form UI (each kind maps to concrete fields), safety (parameterized id-set evaluation, no injection surface, bounded worst-case), and campaign safety — "the preview subtracts exclusions **and the DNC list automatically**, reporting it: '{n} customer(s) removed by the Do-Not-Contact list.'" Unknown kinds parse into a degenerate node that matches nothing (they do not crash). Item 114 is the cautionary tale for why free-form matching lost: "the operator was ignored and the match was unanchored at the tail — every op ran the same `LIKE %@{domain}%`, so `user@example.com.au` matched an `example.com` filter, and `not_equals` / `ends_with` returned the same rows as `equals`." Sharp edge: evaluate-side nesting stops at depth 16 with a note, looser than the parse-side 10 — don't "fix" one without the other.

CHAPTER 25 · PART V

Outreach and incidents

The only feature that emails customers en masse, and the coordination objects that hang off it.

Outreach writes to a real shared Help Scout account, which is why it is the most defensively-built feature. `MAX_ATTEMPTS = 3` exists because of audit item 16: "every recorded outcome is a spent attempt … without this increment it stayed 0 forever, so 429/5xx answers were retryable without limit and the same stuck recipients filled every batch (infinite retry storm + head-of-line block)." The status machine (draft/queued/sending/paused/completed/cancelled) has guard-per-transition; cancel "zeroes only the future: '{cancelled} remaining recipient(s) cancelled. Sent recipients are untouched.'"; manual retry resets the budget visibly. The DNC list is a legal-safety feature — removing someone from it deserves the scariest confirm dialog in the app ("Customer {id} becomes contactable by every campaign again."). And because a desktop app *will* crash mid-send, the mirror (not the local row) is the truth of "did it actually go out": `campaign_reconcile` resolves stranded rows against the platform — "Pure state repair: no continuation job is enqueued here," so

it can never multiply sends. Sharp edge: reconcile *also* completes a campaign when nothing remains; it is not a dry run.

25.1 Incidents: coordination objects, not tickets

Incident codes are `INC-{:03}` from MAX+1 inside the create transaction — stable and gap-tolerant, "what humans quote in notes, known-issue articles and customer-safe explanations." The UI is deliberately create/annotate/link only: an incident is a *coordination object* (which customers, which conversations, what workaround), its lifecycle mostly ends in a known issue, and "a full edit UI invites rewriting history that the timeline (`incident_events`, every mutation recorded with a dedup key) is supposed to preserve." Cluster/known-issue linkage is what lets the Issue Radar auto-create incidents from spikes. `patch_incident` and `update_incident_status` exist in core with no UI caller — if you want resolve buttons, the plumbing is already there and records events.

25.2 Why poll-based sync, not webhook-first

The mirror is the product: search, reports, SLA, segments, AI evidence all run over local SQLite, instantly and offline. Polling with `modifiedSince` + a 10-minute overlap window gives idempotent, resumable, rate-limit-friendly sync (the client honors `x-ratelimit-*-minute`) with no tunnel and no public endpoint; missed webhook events self-heal on the next poll. Webhook ingress is accepted only where the platform gives no alternative — Beacon chats, whose module doc explains: "There is **no list endpoint** — chats arrive through the five `beacon.chat.*` webhook events … The local mirror therefore keeps ended chats (the webhook's last word)." Resources with no list endpoint were removed from the sweep on principle (R6, "honest stubs") rather than faked.

CHAPTER 26 · PART VI

The daily build

A pinned compiler, a size-first release profile, locked dependency graphs, and gates that run before anything expensive.

`rust-toolchain.toml` pins channel `1.99.1` exactly ("Exact pin (D1/D5: stability first). Upgrade deliberately, never silently."), and the workspace's `rust-version = "1.99"` comes with an honesty note: "Older compilers are never tested, so claiming 1.85 would be dishonest." Every cargo call in CI and on the release path carries `--locked`; the `cargo xtask` alias injects it too. The release profile is `codegen-units = 1, lto = true, opt-level = "s", strip = true` — "Small release binaries. LTO + opt-level s + stripped symbols" for the AppImage download use-case.

The dev profile sets `debug = 0`: "keeps memory/disk footprint low when compiling … in constrained environments" (4 GB-class machines were the real build target).

> **Lore correction, for the record**
>
> Outside documents sometimes claim a `-j2` build flag or a thin/fat LTO split between local and CI. The repo contains neither — no `-j` flag, no thin-LTO override, anywhere (grep-verified). Fat LTO applies to every release build. The documented memory levers are `debug = 0`, the slim tokio/reqwest feature sets, and — added by the 2026-10 audit — `CARGO_INCREMENTAL=0` on constrained machines. If a build box struggles, adjust its environment; do not add repo flags that fork the build story.

26.1 The gates, in the order CI runs them

1. `scripts/gate_no_simulation.py` — the no-simulation law, enforced on source (test regions stripped by a brace-counting parser with an allowlist that carries written justifications).
2. `cargo fmt` and clippy twice (with and without the sandbox feature).
3. `cargo test --workspace` twice (same feature matrix).
4. `cargo deny check advisories licenses bans` — and note the audit item 161 lesson: "bans were configured once but never evaluated anywhere." The check must not merely exist; it must run. Duplicate crate versions fail the build until deduped or given a *reasoned* skip line in `deny.toml` ("a maintainer either dedupes (preferred) or adds a documented skip line here").
5. `cargo xtask verify-icons` — "the check had no automation caller before" (item 164); now icon corruption fails before any expensive lane.

Behind all of it sits a process gate: `ai/rule.md` governs the plan→progress→completed workflow the maintainer used (ten items at a time, no silent skips) — the reason the codebase is full of item numbers you can trace.

CHAPTER 27 · PART VI

Packaging

A hand-rolled packager, a dependency list ldd cannot see, and a checksum-pinned tool that builds the AppImage.

`crates/xtask/src/package.rs` produces exactly two artifacts from one release binary: a type-2 AppImage ("self-contained, runs without installation") and a standard

.deb ("declares the GUI toolkit's runtime library set so APT installs everything a desktop session needs"). Packaging "stays in testable halves: `stage_appdir` / `stage_deb` build the staging trees on disk; unit-tested directly." — determinism you can unit-test was worth more than a bundler's convenience. cargo-deb and the Tauri bundler were rejected: neither expresses the dlopen dependency set for APT nor the SHA-pinned toolchain-of-tools this pipeline needs. Naming is a fixed trio because three registries reject different spellings: `SupportOS+` (display), `SupportOS-Plus` (identifier), `supportos-plus` (Debian — "Cargo names cannot carry +").

27.1 The dlopen problem, and the crash that proved it

`package.rs:122-132`: "The GUI stack loads its windowing and graphics libraries at runtime (winit's X11 backends, xkbcommon, glutin/glow, wgpu's GLES fallback and the Wayland client all `dlopen` their system libraries), so **ldd cannot see them**. The .deb therefore declares the client-library set explicitly and APT pulls it in on minimal systems; the GL driver itself stays host-provided (Mesa software rendering works on GPU-less machines)." This is not theoretical: the 2026-10 external audit crashed exactly this way twice — "*crash — xkbcommon-dl panic: libxkbcommon-x11.so could not be loaded*" — before the remediation. Hence the xkb trio bundled into the AppImage: "winit loads the X11 keyboard stack via dlopen at window creation (xkbcommon-dl), and not every Linux host ships the X11 flavor … The trio is MIT/X11-licensed, glibc-free and a few hundred KiB." If the tool cannot bundle xkb, the build fails with an apt-oriented hint — "shipping an AppImage that cannot open a window would be worse than failing the build."

27.2 Supply-chain discipline

The appimagetool binary executed on the release path is version+SHA-256 pinned (clean-audit item 158): "The URL used to point at the moving `continuous` build with no integrity check — whatever GitHub served that day was executed in the release path. The version tag and SHA-256 are pinned instead: a mismatched tool (tampered download, corrupted cache, upstream re-upload) refuses to run." The download itself is bounded and atomic (item 160): timeouts, a `.part` sibling, fsync, rename — "a crash mid-download used to leave a truncated file at the cached path itself." Payload ownership is stamped `root:root` and mode-normalized "because the build user's umask (e.g. 002 in CI containers) must not leak into the installed package"; there are *no* maintainer scripts — "the desktop-file and hicolor caches are maintained by their system trigger providers … and user data lives under

XDG_DATA_HOME where the package never owns it." Upgrading appimagetool means changing the URL *and* the SHA constant, together, deliberately.

CHAPTER 28 · PART VI

The release pipeline

Tag in, verified artifacts out: a clean container, a three-distro matrix, and a publish step that re-downloads its own assets.

`.github/workflows/release.yml` triggers on `v*` tags. Its shape is worth walking once, top to bottom:

1. **Queue, never cancel.** Release runs serialize for the same ref with `cancel-in-progress: false` — "a half-published release is worse than a queue." (CI cancels; release queues. Both are correct.)
2. **Clean container = compatibility contract.** The build runs in `ubuntu:22.04` "because the glibc the binary links becomes the floor for every user; building on the oldest supported distro guarantees the artifacts run on 22.04+, Debian 12+, and newer." Building on `ubuntu-latest` would silently raise the floor.
3. **Gates first** — the no-simulation gate ("a tagged release must not carry fabricated-data markers"), fmt, clippy ×2, tests ×2, all `--locked`. The toolchain action reads `rust-toolchain.toml` as "single source of truth".
4. **Build once, package twice.** One release build with the sandbox feature; then `package --skip-build` and `deb --skip-build` ("Skips the cargo build — already built, e.g. CI re-runs"). Both artifacts come from the same bytes that were tested.
5. **Provenance files.** `SHA256SUMS.txt` plus `BUILD-INFO.txt` (product/version/commit/built-on/built-at) — the link between artifact and commit, enforced later by a literal grep: `grep -q "commit: $SHA" dist/BUILD-INFO.txt || exit 1`.

6. **Verify matrix, three distros** (ubuntu 22.04/24.04, debian 12): checksum re-verify; `apt-get install ./….deb` — "must resolve and fetch every declared dependency from the distribution repository — no manual dependency handling"; metadata + desktop-file validation; `SPP_SMOKE=1` boot smoke ("verifies logging, single-instance lock, all boot migrations, boot services, background workers and the loopback listener, then exits 0 without opening a window"); a scripted 0.2.x→1.x data-directory migration; a GUI launch under Xvfb where "every client library must come from the .deb's own Depends set" (the net that catches a missing dlopen lib); reinstall, uninstall-keeps-user-data, purge. The AppImage job verifies via `APPIMAGE_EXTRACT_AND_RUN=1 SPP_SMOKE=1`, which "validates paths/icons/AppRun without FUSE."
7. **Publish, last, gated, then re-verified.** Release notes are composed and the CHANGELOG section is extracted by an awk contract that expects Keep-a-Changelog headings `## [X.Y.Z]` — "Missing section ⇒ notes stay as before (no-op)" (so breaking the heading format fails silently — don't). Uploads clobber on re-runs because "re-built artifacts carry fresh tar timestamps; the notes' checksum table must always match the uploaded bytes." Then the pipeline **re-downloads every asset and re-checks its SHA-256**. Nothing is published unless every step passed.

> **Lore correction #2**
>
> The smoke step is `SPP_SMOKE=1 ./…` with the container's own `HOME` — fresh containers make the data dir clean anyway; the 0.2.x-upgrade step *constructs* a legacy dir deliberately. Scratch `XDG_DATA_HOME` workarounds belong to audit-era local runs, not the checked-in workflow. The workflow file is the source of truth.

CHAPTER 29 · PART VI

The history rewrite

The one piece of repo lore you cannot learn from the code, because the code's history was deliberately changed.

`repo-audit-2026-10-24/HISTORY-REWRITE.md` records that, with the maintainer's authorization (FIND-003), eight QA-fixture `secrets.key` files were purged "from **every commit of every ref**". Because "git tag --contains proved **all 11 release tags** descend from the secret-introducing commits," the tags were rewritten together

with main (git filter-repo; the old→new SHA map ships as `HISTORY-REWRITE-ref-map.txt` and `commit-map.txt`). Four consequences for you:

- **Old SHAs will not resolve.** Any hash quoted in commit messages, audit docs, or release notes is pre-rewrite; translate it through the ref map. Commit *messages* were intentionally left with historical values ("messages are historical narrative and were not rewritten").
- **Pre-rewrite backup bundles still contain the purged material** and must stay local-only, forever.
- **Never force-push casually again.** The tags are the only release identity; the history is now clean by design.
- **The forward guards are load-bearing:** a `.gitignore` rule plus two xtask hygiene tests ("no file named secrets.key anywhere in the tree"; "the .gitignore rule must stay in place") make re-tracking a secrets file a CI failure. Do not remove them.

CHAPTER 30 · PART VII

The laws of the codebase

Every invariant from Parts I–VI, compressed into one list. If a change violates a law here, the change is wrong until the law is consciously amended.

30.1 Platform and data

- Never edit an applied migration. New schema = new `BootStep` version 54+ with an idempotent apply function (the m039/m062 outage is the proof).
- Every connection opens via `db::open` — the four pragmas are the contract; a raw `Connection::open` silently drops FK enforcement.
- Reads via `with_read`, short writes via `with_write`, network-spanning work via `long_op_guard`. Never hold a DB guard across an `.await` or a network call.
- Mutex poisoning is recovered, never propagated; every timer gets a `TickGuard` and runs inside `contained()`.
- The UI thread never blocks. Spawn; report via the bus; push events *before* repainting; register every new `ServerEvent` in `pages::invalidated_pages`.
- Everything heavy lives in the library; `main` only boots. The smoke constructs the real shell from a bare `egui::Context`.
- No simulated/demo/fake data, ever — the CI gate runs first; boot is `Real` or `Empty`; empty→real transitions reset the sync watermark.

- `catalog` stays I/O-free and unsafe-free; `core` stays UI-free; the ladder never gains a cycle.
- `unsafe` exists in exactly two places: the two flock FFI calls, and the sqlite-vec FFI registration. Nowhere else.
- Env vars are the only configuration surface (no CLI args). Toast lifetimes burn on rendered passes; only manual syncs toast.
- The flock is acquired before the DB opens and held for process lifetime; a second instance exits 1.
- User data is never package-owned; uninstall/purge must not touch the data dir.

30.2 Security and data safety

- Never store a credential in plaintext; every secret goes through `set_secret` (the envelope). The OAuth token columns stay permanently empty.
- Never log, Debug-print, or toast a secret value; log key names and presence only.
- Every write of customer data is chmod'd 0600 immediately (`crypto::restrict_permissions`).
- Never touch `secrets.key`; it is created once and refuses to be clobbered.
- Never swap a file over the live DB without verify-first, and move live db+wal+shm aside *as one unit*. Restore replaces; it never merges.
- Never accept an unsigned webhook; registration and verification resolve the secret through one function; duplicates never mutate the original row.
- Never run an untrusted-document parse outside the sandbox in release builds; sandbox closures take moved-in bytes only — no paths, no sockets.
- Never render remote HTML without `sanitize_thread_html`; never run the CSS blocklist on undecoded values.
- Every new user-configured URL fetch runs the SSRF guard (async variant on async paths) and uses `Policy::none()`.
- POSTs are never auto-retried after ambiguous failures. No background/periodic network calls — the version check is user-initiated, full stop.

30.3 The AI stack

- Never make an agent tool write. New tools are SELECTs over the mirror, redacted and clamped.

- Never send a draft without its evidence pack and its verification pass; verification parse failure means `verified = false`.
- Never strip the code-enforced customer-safe filter from the draft renderer; never feed quarantined memory into a prompt.
- Never cross the redaction boundary with raw customer text.
- Never cache or persist an analysis that fails the core-field gate; never record a run without completing or failing it.
- Never enqueue a golden eval per keystroke; never make eval failure block a role-map save.
- Never change chunk geometry (1200/150) without planning a full re-embed; never upsert vectors with raw per-table ids; never assume embeddings are normalized.
- Never remove the honest-fallback semantics of retrieval — degraded lanes print their mode note.
- Every new AI surface gets an `AI_FEATURES` row (on-demand ⇒ ON, automatic generative ⇒ OFF) and the kill switch covers it.
- Prompt edits are version bumps — the cache index and the golden fingerprint depend on it.

30.4 Product and release

- Destructive actions are two-phase (confirm dialog → armed → execute → honest outcome toast); a cancel leaves the world untouched.
- Every new table needs a conscious KEEP decision in `data_reset.rs` (the pinned test makes forgetting it a build failure).
- Nothing fabricates report data; capability banners drive every UI claim; segments are trees, never SQL.
- Every cargo call on the release path is `--locked`; the toolchain pin upgrades as deliberate commits; duplicate crate versions need dedupe or a reasoned skip.
- Both packages carry the sandbox feature; both artifacts come from the same release binary; the appimagetool URL and SHA change together or not at all.
- Build releases in the ubuntu 22.04 container — the glibc floor is the compatibility contract. Never cancel in-progress release runs.
- CHANGELOG headings stay `## [X.Y.Z] — date` — the release-notes extractor depends on the bracketed heading.

- `secrets.key` is never tracked again; the hygiene tests stay; old SHAs go through the ref map.

CHAPTER 31 · PART VII

Sharp edges A–Z

"If you are about to …, read this first." The quick-reference table of changes that have bitten before, or will bite predictably.

IF YOU ARE ABOUT TO…	STOP AND READ THIS
Add a database table or column	New `BootStep` v54+; idempotent SQL; KEEP decision in `data_reset.rs`; never edit v≤53 (Ch. 7, 12).
Add a UI page	It is automatically smoke-covered — it must paint non-zero shapes; register its confirm dialogs via `NavRequest::Confirm` (Ch. 4, 30).
Add a background event or timer	`invalidated_pages` map; TickGuard + contained(); workers: None in tests (Ch. 5, 8).
Add an AI surface	`AI_FEATURES` row; on-demand ON, automatic generative OFF; execution-time gate; kill switch covers it (Ch. 16).
Swap the embedding model	`ensure_collection(new_dim)` rebuild + pending re-embed; dimension mismatch fails loudly by design (Ch. 20).
Change chunk size or embed batching	Full re-embed required; no chunk_version exists; partition per chunk, attempts cap 5 (Ch. 20).
Edit a prompt	Bump its version constant; cache + golden fingerprint depend on it; never clean old `ai_runs` versions (Ch. 21).
Add an agent tool	Read-only SELECT only; redact output; clamp limits; add to `CopilotTool::ALL` (Ch. 19).
Add any outbound HTTP client	Explicit redirect policy (`Policy::none()`), SSRF guard (async twin on async paths), timeout; five egress points exist today (Ch. 15, 16).
Add a dependency	Duplicate crate versions fail `cargo deny` until deduped or reasoned-skip; licenses into the allowlist; `--locked` everywhere (Ch. 26).
Upgrade eframe/egui	Exact pin `=0.37` is deliberate; renderer fallback chain and fatal.log context are load-bearing; charts stay on the painter until egui_plot catches up (Ch. 3, 26 of the platform dossiers).

Upgrade appimagetool	URL + SHA-256 constants change together; cached tools re-verify (Ch. 27).
Add a destructive action	Two-phase confirm via the shell dialog; honest outcome toast; cancel touches nothing (Ch. 30).
Add an export artifact	`restrict_permissions` immediately; sidecar paths change extension only (M18); redact secret-shaped keys (Ch. 12).
Parse a new untrusted format	Route through `run_sandboxed`; moved-in bytes only; caps before allocation (Ch. 10, 15).
Touch restore or import	Verify-first; move db+wal+shm as one unit; replace, never merge; copy the ordering from `backup_service.rs` verbatim (Ch. 12).
Add an OAuth/loopback path	Single-UPDATE state consume; 10-min TTL; port-bound redirect; tokens only into the envelope (Ch. 14).
Add an automation trigger or fire path	Replicate the once-guard consumption discipline; respect the double kill switch; approved writes need the outbound push (Ch. 22).
Change the CHANGELOG format	Keep-a-Changelog `## [X.Y.Z]` headings or the release-notes awk extraction silently no-ops (Ch. 28).
Add a non-Linux package target	The sandbox feature is Linux-only and forced by `cfg!(target_os = "linux")` — a non-Linux artifact would ship sandboxless (Ch. 10, 27).

CHAPTER 32 · PART VII

Known trade-offs, open ends

The honest register: accepted risks, dormant paths, deferred features — so the next maintainer inherits knowledge, not surprises.

32.1 Accepted risks (documented, deliberate)

- **SSRF re-binding window.** The guard resolves DNS once; a rebinding server can rotate between check and fetch. Accepted for a local desktop tool.
- **secure_delete ≠ media erasure.** Pages and WAL are zeroed; SSD wear-leveling remnants are not addressed. Attachments are removed, not overwritten.

- **Unbounded growth by default.** `ai_runs` retention is off unless the user sets it; `audit_log` (action history) is uncapped by design — only the noisy diagnostics ring is capped at 500.
- **Verifier quality.** Draft verification is itself a model call; a weak verifier can rubber-stamp. Mitigations: fail-closed parse handling, visible verification outcome, human send gate.
- **Eval ceiling.** Golden evals measure parseability and core-field presence, not factual accuracy.
- **Chunk tails.** Thread bodies are pre-truncated to 4000 chars before vector-space embedding, though FTS sees the full text.
- **English-only quarantine.** The memory guard's vocabulary is English; paraphrases and other languages pass by design.
- **Loopback port collision.** If port 3000 is taken, webhook ingress degrades with a log warning plus a UI badge (FIND-001); conversations sync by polling regardless.
- **Pre-rewrite bundles.** Old local backup bundles still contain purged key material; they stay local-only (Ch. 29).

32.2 Dormant paths (exist, but nothing calls them)

- `Trigger::SlaRisk` is selectable in the automation rule form; no code currently emits the event.
- `patch_incident` / `update_incident_status` exist in core with no UI caller — resolve buttons are plumbing-complete.
- `COACHING_MAX_DRAFT_CHARS` (20k) is declared but unreferenced; the effective caps are the prompt slices (6000 chars into the AI, 240-char excerpts stored).
- A stale comment mentions an "embedded qdrant engine"; sqlite-vec replaced it (D4) and the `VectorStore` trait keeps the swap reversible.
- egui_plot does not target egui 0.37 yet; charts ride the shell's own painter (`chart.rs`) until then — re-pin when upstream catches up.

32.3 Deferred by design (recorded scope cuts)

- **Second-instance forwarding.** The flock refuses and exits; "focus the existing window" needs IPC that does not exist yet — the comment says exactly this.
- **PKCE.** State + loopback + envelope-stored secret cover the local threat model; adding PKCE is additive if you ever want it.

- **Cross-platform packaging and mobile.** Declared non-goals; the aarch64 mapping exists in the packager but release stages x86_64 names only.
- **Scheduled triggers / cron automation.** Every scheduled concept is computed at sync/sweep time; a timer layer would need wake-the-app and catch-up semantics.

32.4 Where the ledgers live

When this book and the code disagree, re-read the code; when the code is silent, the ledgers are the next stop: `ai/decisions.md` (the port's decision ledger and migration map), `ai/plan.md` and `ai/completed.md` (the clean-audit item ledgers), `ai/rule.md` (the working process), and `repo-audit-2026-10-24/` (the external audit, its findings, and the history-rewrite record). The item-numbered comments in the source are the crosswalk between them.

One last thing. The most important habit you can inherit is not in any chapter: **when you fix a bug, leave a comment that explains what it used to do wrong**. That is why this book exists — the previous maintainer wrote the first draft of it, one `Audit fix (M…)` comment at a time. Keep writing it.

The code is the documentation. The comments are the history. This book is the map.

SupportOS+ Developer Handover Book — written from the source, for the next maintainer.

VERSION 1.8.0 · KIMPEARCE888/SUPPORTOS-PLUS